실습 · 시승 예약 MVP

시승 예약 웹앱 만들기

Stitch와 Claude Code로 화면 디자인부터 예약 저장과 테스트까지

실습 키트의 데이터와 규칙 문서만으로 끝까지 따라간다

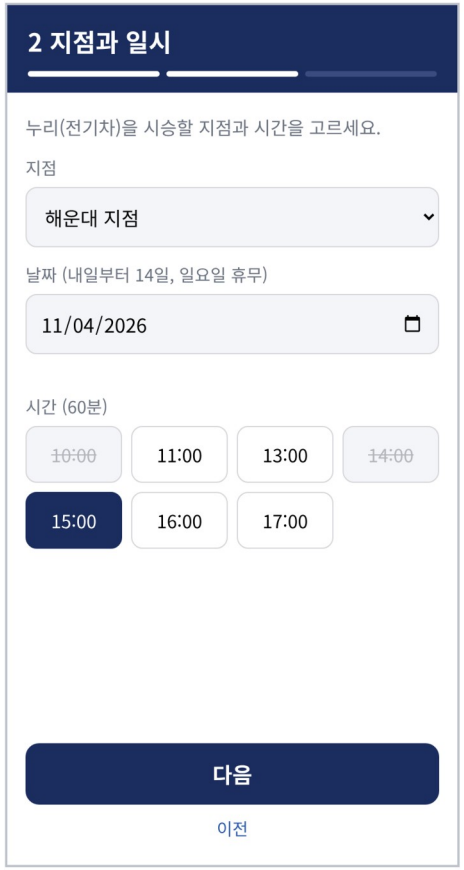
데스크톱 앱 Code 탭 기준입니다. 모든 단계는 프롬프트 한 장으로 시작하고, 결과 화면과 막혔을 때의 대처를 함께 보여 줍니다. 회사명과 차종명, 주소는 모두 실습용으로 지어낸 것입니다.

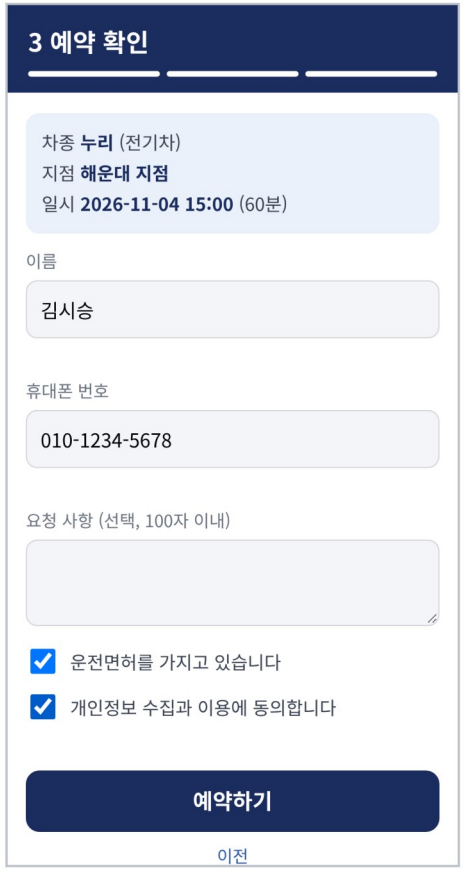
오늘 만들 것

세 화면을 지나면 시승 예약이 끝나는 웹앱을 만든다

차종을 고르고, 지점과 시간을 고르고, 정보를 넣으면 예약번호가 나온다









오늘 쓰는 용어 네 개를 먼저 정리한다

처음 듣는 말이 나오면 이 표로 돌아온다

용어	뜻	오늘 어디에 쓰나
MVP (최소 기능 제품)	꼭 필요한 기능만 넣어 먼저 돌려 보는 첫 버전	세 화면짜리 예약 웹앱
웹앱 (HTML 파일)	브라우저에서 열리는 프로그램. 오늘은 파일 하나다	testdrive_mvp.html
Stitch MCP	클코가 Stitch에 화면을 만들라고 시키는 연결 통로	1단계 화면 디자인
localStorage	브라우저 안에 작은 기록을 남겨 두는 저장 공간	5단계 예약 저장

Stitch는 겉모습을, 클코는 속을 맡는다

Stitch가 그린 화면에 클코가 데이터와 규칙을 넣어 한 파일로 완성한다



01

PART 1 · 실습 키트

데이터와 규칙을 먼저 읽는다

무엇을 만들지 모르는 채로 프롬프트를 쓰지 않는다

키트에는 데이터 세 개와 문서 두 개가 들어 있다

testdrive_kit 폴더를 Code 탭의 작업 폴더로 연다

파일	내용	쓰는 단계
docs/design_brief.md	화면 디자인 요청서	1단계
docs/booking_rules.md	예약 업무 규칙	4~6단계
data/vehicles.json	차종 네 개	3단계
data/branches.json	지점 세 곳과 지점별 보유 차종	4단계
data/bookings_sample.json	이미 들어온 예약 네 건	4단계
images/stitch_screens_example.png	세 화면 배치 예시 그림	1단계
reference/..._reference.html	완성본 예시	7단계

차종은 성격이 다른 네 대로 구성했다

data/vehicles.json · 가격 단위는 만 원

모델	차종	연료	승차	효율	시작 가격
온유	세단	가솔린	5인	복합연비 13.8km/L	3,450
바람	SUV	하이브리드	7인	복합연비 15.2km/L	4,980
솔	해치백	가솔린	5인	복합연비 15.9km/L	2,390
누리	전기차	전기	5인	1회 충전 450km	5,290

카드 한 장에는 차 그림 자리와 모델명, 차종과 연료, 시작 가격이 들어간다

지점마다 시승할 수 있는 차가 다르다

data/branches.json · 2번 화면은 1번에서 고른 차가 있는 지점만 보여 준다

	온유 세단	바람 SUV	솔 해치백	누리 전기차
해운대 지점	✓	✓	없음	✓
서면 지점	✓	없음	✓	없음
센텀 지점	없음	✓	✓	✓

누리(전기차)를 고르면 해운대와 센텀 지점만 보인다

업무 규칙

시간은 60분 단위이고 이미 찬 시각은 막는다

booking_rules.md 3장과 4장 · bookings_sample.json의 예약이 흐린 칸이 된다

해운대 지점에서 누리(전기차)를 고른 첫 번째 영업일의 시간표

10:00	11:00	점심	13:00	14:00	15:00	16:00	17:00
이미 예약	예약 가능	받지 않음	예약 가능	이미 예약	예약 가능	예약 가능	예약 가능

한 번에 60분

시작 시각은 일곱 개
12시는 점심시간

이미 찬 시각

같은 지점, 같은 차,
같은 시각이면 막는다

날짜 범위

내일부터 14일까지
일요일은 휴무

예약자 정보는 네 칸이 필수다

booking_rules.md 5장과 6장 · 틀리면 그 칸 바로 아래에 빨간 글씨로 알린다

항목	필수	확인할 것
이름	필수	두 글자 이상
휴대폰 번호	필수	010-0000-0000 형식
운전면허 보유	필수	체크해야 예약된다
개인정보 수집 동의	필수	체크해야 예약된다
요청 사항	선택	100자 이내

예약번호

- TD-날짜-네 자리 숫자
- 예: TD-20261104-5422
- 완료 화면에 지점 전화번호도 함께 보여 준다

전체 흐름

여덟 단계로 나눠 한 단계씩 만든다

한 번에 다 시키지 않는다. 단계마다 결과를 눈으로 확인하고 다음으로 간다



02

PART 2 · 구현

프롬프트 한 장씩, 화면 하나씩

0단계부터 5단계까지. 단계마다 결과 화면과 피드백을 확인한다

먼저 클코에게 키트를 읽히고 할 일을 요약하게 한다

만들기 전에 이해했는지 확인한다. 요약이 틀리면 여기서 바로잡는다

[입력] 프롬프트 · Claude Code (Code 탭)

```
이 폴더의 README.md, docs 폴더,
data 폴더를 모두 읽고 오늘 무엇을
만들어야 하는지 다섯 줄로 요약해줘.
아직 아무 파일도 만들지 마.
```

피드백

- 요약에 세 화면과 일요일 휴무, 마감 시각 처리가 빠져 있으면 다시 읽으라고 한다

[출력] 클코의 요약 (예시)

1. 세 화면짜리 모바일 시승 예약 웹앱
2. 지점마다 보유 차종이 달라 고른 차가 있는 지점만 보여 줌
3. 내일부터 14일, 일요일 휴무, 60분 단위 일곱 개 시각
4. 이미 찬 시각은 흐리게 막음
5. 입력 확인 후 예약번호를 만들고 localStorage에 저장

디자인 요청서를 넘겨 Stitch에 화면 세 장을 만든다

Stitch MCP가 연결돼 있어야 한다. 안 돼 있으면 다음 장의 대안을 쓴다

[입력] 프롬프트 · Claude Code (Code 탭)

```
docs/design_brief.md 내용대로
Stitch에 testdrive-booking 프로젝트를
만들고 모바일 화면 3개를 생성해줘.
배치는 images/stitch_screens_example.png
를 참고해. 다 만들면 화면마다
무엇이 들어갔는지 알려줘.
```

피드백

- Stitch 웹사이트에서도 같은 화면이 보이는지 확인한다
- 고칠 곳은 '2번 화면 버튼을 더 크게'처럼 말로 지시한다

[출력] Stitch가 만든 화면 (배치 예시)



Stitch 연결이 없어도 클코가 같은 화면을 그릴 수 있다

디자인 요청서와 예시 그림만 있으면 된다. 이후 단계는 똑같이 진행한다

[입력] 프롬프트 · Claude Code (Code 탭)

```
Stitch는 쓰지 말고, docs/design_brief.md와
images/stitch_screens_example.png를 보고
화면 3개를 HTML로 직접 디자인해서
stitch 폴더에 screen-1.html, screen-2.html,
screen-3.html로 저장해줘.
```

언제 대안을 쓰나

- Stitch API 키가 아직 없을 때
- /mcp 목록에 stitch가 connected로 안 보일 때

Stitch를 연결하려면

- Stitch 설정에서 API 키를 만든다
- 클코에게 키 자리를 비운 등록 명령어만 받는다
- 키를 채워 터미널에서 실행한다
- 앱을 재시작하고 /mcp로 확인한다

세 화면을 파일 하나로 합치고 버튼으로 넘긴다

아직 데이터와 규칙은 넣지 않는다. 화면이 순서대로 넘어가는지만 본다

[입력] 프롬프트 · Claude Code (Code 탭)

```
Stitch에서 만든 화면 3개의 HTML을
stitch 폴더에 저장해줘. 그다음 세 화면을
testdrive_mvp.html 한 파일로 합치고
'다음'을 누르면 다음 화면으로 넘어가게 해줘.
데이터와 기능은 아직 넣지 마.
다 되면 브라우저로 열어줘.
```

피드백

- 화면이 한꺼번에 다 보이면 '한 번에 한 화면만 보이게 해줘'라고 한다

[출력] 폴더 구성

```
testdrive_kit/
├── stitch/
│   ├── screen-1.html
│   ├── screen-2.html
│   └── screen-3.html
├── testdrive_mvp.html   ← 새로 생김
├── data/  docs/  images/
└── reference/
```

차종 데이터로 1번 화면의 카드를 채운다

data/vehicles.json의 네 대가 카드가 된다

[입력] 프롬프트 · Claude Code (Code 탭)

```
data/vehicles.json의 차종 네 개로
1번 화면의 카드를 채워줘. 파일을
더블클릭해도 열리도록 데이터는
HTML 안에 넣어줘. 아무것도 안 고르고
'다음'을 누르면 카드 아래에 빨간 글씨로
"차종을 하나 고르세요."라고 알려줘.
```

피드백

- 데이터를 HTML 안에 넣는 이유: 브라우저는 파일로 연 페이지가 옆의 json 파일을 읽지 못하게 막는다

[출력] 결과 화면

안 고르고 다음



누리를 고름



지점과 날짜, 시간 규칙을 2번 화면에 넣는다

booking_rules.md 2장부터 4장까지 · 해운대 지점 누리의 10시와 14시가 막혀야 한다

[입력] 프롬프트 · Claude Code (Code 탭)

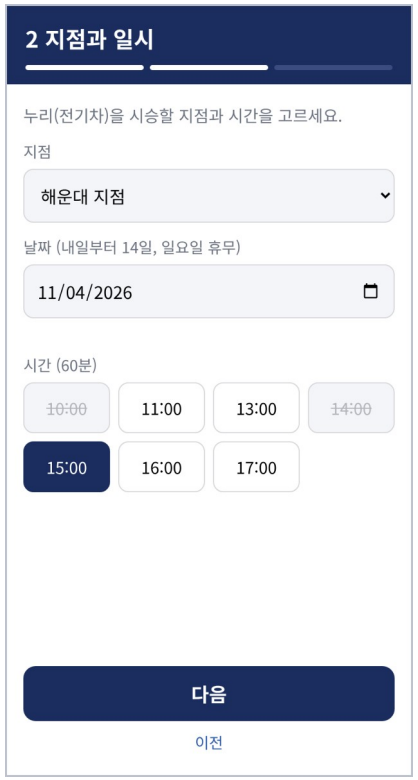
```
docs/booking_rules.md의 2장, 3장, 4장대로
2번 화면을 동작하게 해줘. 고른 차가 있는
지점만 보여 주고, 날짜는 내일부터 14일까지
고르게 하되 일요일은 막아줘. 시간은
일곱 개 버튼으로 보여 주고, 이미 있는
예약 시각은 흐리게 해서 못 누르게 해줘.
```

피드백

- 누리를 고르고 해운대 지점에서 10시와 14시가 흐린지 본다
- 일요일을 고르면 경고가 나오는지 본다

[출력] 결과 화면

마감 시각이 흐림



일요일을 고름



localStorage는 브라우저 안의 작은 메모장이다

서버 없이도 새로고침 뒤에 예약이 남아 있게 해 준다

할 수 있는 것

- 새로고침하거나 창을 닫았다 열어도 기록이 남는다
- MVP에서 서버 없이 저장을 흉내 낼 수 있다

한계

- 그 브라우저 안에만 남는다. 다른 사람은 못 본다
- 실제 서비스에서는 서버나 데이터베이스로 옮긴다

오늘은 localStorage로 저장하고, 실제 서비스는 확장 과제로 남긴다

5단계

입력을 확인하고 예약번호를 만들어 저장한다

booking_rules.md 5장과 6장 · 틀린 칸 바로 아래에 이유를 보여 준다

[입력] 프롬프트 · Claude Code (Code 탭)

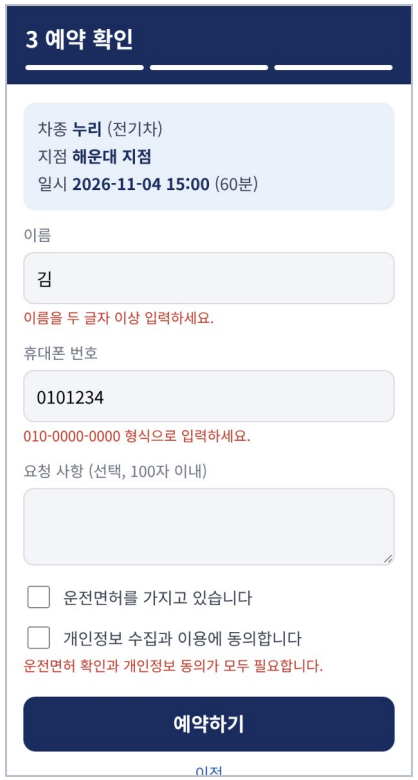
```
docs/booking_rules.md의 5장과 6장대로
3번 화면의 입력 확인과 완료 화면을
만들어줘. 틀린 칸 바로 아래에 이유를
빨간 글씨로 보여 줘. 예약은 localStorage에
저장해서 새로고침해도 그 시각이
막혀 있게 해줘.
```

피드백

- 이름 한 글자, 하이픈 없는 번호, 체크 안 함을 일부러 넣어 본다

[출력] 결과 화면

틀리게 넣음



예약 완료



5단계 확인

새로고침 뒤에 방금 예약한 시각이 막혀 있어야 한다

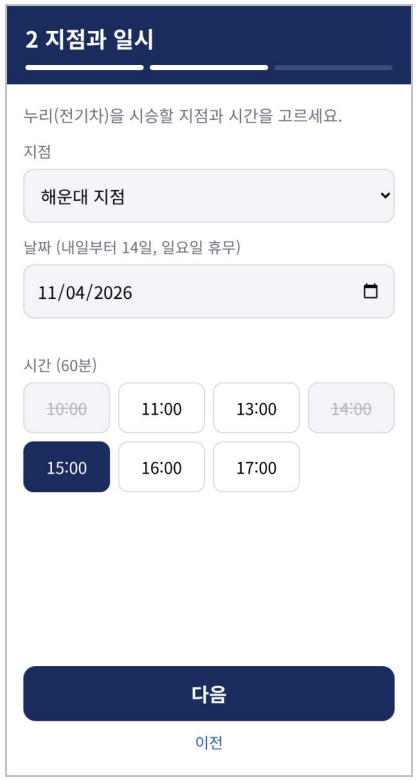
누리를 해운대 지점에서 15시에 예약한 뒤 새로고침해 같은 날을 다시 연다

확인하는 법

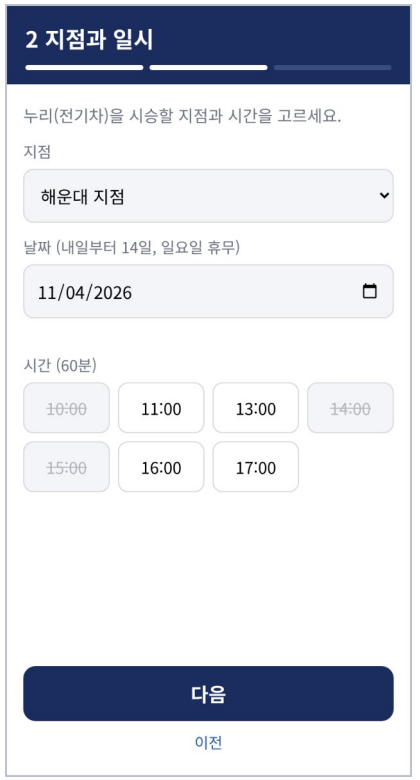
- 누리, 해운대 지점, 첫 영업일 15시로 예약한다
- 브라우저를 새로고침한다
- 같은 차와 지점, 날짜를 다시 고른다
- 15시가 10시, 14시처럼 흐리면 저장이 된 것이다

[출력] 새로고침 전후

예약 전 (15시 고름)



새로고침 뒤



03

PART 3 · 확인

규칙대로 동작하는지 증명한다

눈으로 대충 보지 않고, 규칙마다 테스트를 돌려 표로 남긴다

규칙마다 테스트를 돌려 결과를 표로 받는다

클코가 브라우저를 직접 눌러 보고, 실패하면 고친 뒤 다시 확인한다

[입력] 프롬프트 · Claude Code (Code 탭)

```
docs/booking_rules.md의 규칙마다
테스트 시나리오를 하나씩 만들고
브라우저로 직접 눌러 보며 확인해줘.
결과는 규칙, 해 본 것, 결과 세 칸 표로
보여 주고, 실패한 것은 고친 뒤
다시 확인해줘.
```

피드백

- 실패가 하나도 없다고 하면 '일요일 테스트는 어떻게 했어?'처럼 하나를 골라 되묻는다

[출력] 테스트 결과 (예시)

규칙	결과
지점 필터	통과
일요일 차단	통과
마감 시각 흐림	통과
번호 형식	실패 → 고침
저장 유지	통과

완성본과 같은 순서로 눌러 동작 차이를 찾는다

디자인은 달라도 된다. 규칙을 다르게 처리하는 곳만 찾는다

[입력] 프롬프트 · Claude Code (Code 탭)

```
reference/testdrive_mvp_reference.html과
내 testdrive_mvp.html을 같은 순서로
눌러 보고, 규칙을 다르게 처리하는 곳이
있으면 알려줘. 디자인 차이는 괜찮아.
```

피드백

- 차이가 나오면 어느 쪽이 booking_rules.md에 맞는지 다시 물어 결정한다

[출력] 비교 결과 (예시)

```
같음: 지점 필터, 일요일 차단,
      마감 시각, 예약번호 형식

다름: 내 앱은 요청 사항이
      100자를 넘어도 저장됨

→ booking_rules.md 5장에 맞게
  100자 제한을 넣을까요?
```

자주 막히는 곳은 다섯 군데다

증상을 보고 해당 단계로 돌아가 프롬프트를 고쳐 다시 시킨다

증상	원인	이렇게 다시 시킨다
카드가 비어 있다	json을 파일 밖에서 읽으려 했다	데이터를 HTML 안에 넣어줘
모든 지점이 보인다	보유 차종을 안 따졌다	branches.json의 vehicles로 걸러줘
마감 시각이 안 흐리다	D+1을 날짜로 안 바꿨다	booking_rules.md 4장을 다시 읽어줘
새로고침하면 사라진다	저장을 안 했거나 안 읽었다	열 때 localStorage를 읽어 반영해줘
Stitch 도구가 없다	MCP 연결이 안 됐다	1단계 대안 프롬프트를 쓴다

확장 과제

MVP 다음에는 직원 쪽 화면과 실제 저장이 필요하다

같은 방식으로 프롬프트 한 장씩 이어서 만든다

과제 1	지점 직원이 오늘 예약을 시간순으로 보는 관리 화면을 만든다	**관리 화면**
과제 2	고객이 예약번호로 예약을 찾아 취소하게 한다	**취소 기능**
과제 3	차 그림 자리에 사용 허락을 받은 실제 사진을 넣는다	**이미지 교체**
과제 4	localStorage 대신 Supabase 같은 데이터베이스에 저장한다	**실제 저장**

규칙 문서가 있으면 프롬프트는 짧아진다

1

만들기 전에 데이터와 규칙을 문서로 정리한다

프롬프트에는 '5장대로'처럼 문서의 장만 가리키면 된다

2

겉모습은 Stitch, 데이터와 규칙은 클코에게 나눠 맡긴다

Stitch가 없어도 같은 요청서로 클코가 화면을 그릴 수 있다

3

단계마다 화면을 확인하고, 마지막엔 규칙별 테스트로 증명한다

실패한 테스트는 고친 뒤 다시 돌려 표로 남긴다

다음 단계: 관리 화면과 실제 데이터베이스 연결